Denoising Diffusion Probabilistic Models for Driving Scenario Generation

Blake Taylor, Fang Fang, Inzaghi Moniaga

March 26, 2026

I. Introduction

What is the most effective way to collect rare driving scenarios for autonomous vehicles? Relying solely on real-world data is dangerous, costly, and slow [Ettinger et al., 2021]. Simulation provides a cost-effective way to capture edge-case data. At any moment, a nearby vehicle with a similar observed history could crash, speed dangerously, or commit an illegal maneuver. To address this uncertainty, a motion forecasting system should not predict only one likely future; it must provide a comprehensive distribution of possible trajectories, including dangerous ones.

Traditional deterministic predictors are poorly suited to this setting, as human behaviour is inherently stochastic and trajectory prediction is fundamentally a probabilistic problem. Generative models offer a more appropriate alternative: rather than outputting one trajectory, they learn a distribution from which diverse and realistic futures can be sampled.

Among generative approaches, diffusion models have recently become especially successful in generating images [Kong et al., 2021, Chen et al., 2021], protein structure [Luo et al., 2022, Trippe et al., 2023, Ingraham et al., 2023], and driving scenario generation [Li et al., 2023, Chen et al., 2023, Wang et al., 2024], suggesting that they are a natural fit for realistic motion forecasting in autonomous driving.

We present a context-conditioned diffusion framework for multi-vehicle agent driving scenario generation using the Waymo Open Motion dataset. Our architecture combines a local-coordinate scene encoder, a conditioned diffusion generator, and a global trajectory decoder. To study the trade-off between predictive accuracy and physical realism, we systematically compare four decoding strategies, from a constant-velocity baseline to a fully safety-constrained bicycle model. The remainder of the paper is organized as follows: Section II defines the forecasting problem and scene representation, Section III introduces the diffusion model and guidance methodology, and Section A details our experimental evaluation of trajectory accuracy and safety.

II. Problem Setup

A. What are we trying to predict?

In our dataset we are given the first 11 time steps of motion data. If we freeze a traffic scene at the present moment, t_0, we have a clear picture of the past and present. We know the recent motion history of every vehicle, the map geometry, and the current positions of any nearby vehicle agents. We define this observed scene context as $\mathcal{C}$.

What we do not know is how the scene will unfold. A single vehicle might continue straight, slow down, or turn. In a multi-vehicle agent scenario, this uncertainty compounds rapidly because every driver's decision alters the behavior of the drivers around them. Therefore, an effective forecasting model cannot assume there is only one correct outcome. It must model a distribution of possible futures conditioned on the known environment.

Our goal is to learn the distribution $p(\mathbf{x}_0 \mid \mathcal{C})$, allowing the model to take the observed context ($\mathcal{C}$) and generate realistic future trajectories ($\mathbf{x}_0$) for every vehicle agent in the scene over the next $F = 80$ timesteps. The goal can be visualized in Figure 1

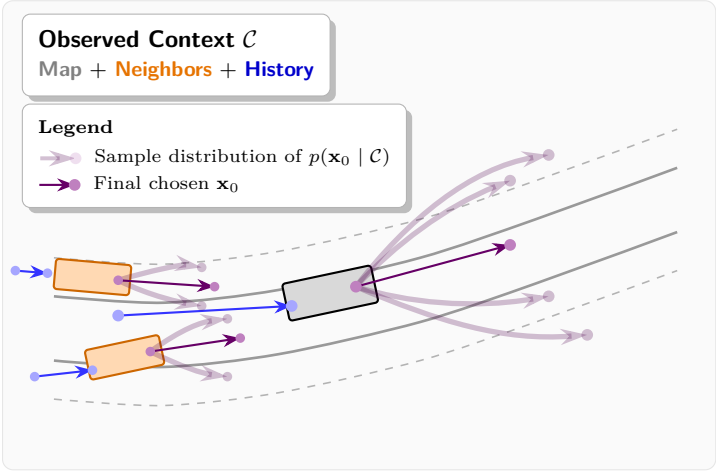

Figure 1: The forecasting task: from observed context $\mathcal{C}$ to a distribution over plausible joint futures.

B. Keeping three notions of time separate

Because our model generates the full future by an iterative denoising algorithm, there are three different notions of "time" in the paper. It is important to separate them clearly:

- **Observation time** t_0: the present moment in the recorded data. This is the fixed anchor from which prediction begins.
- **Forecast time** $f \in \{1, \ldots, F\}$: the physical future timestep we want to predict.
- **Diffusion time** $t \in \{0, \ldots, T-1\}$: the internal step index of the generative model's reverse denoising algorithm.

In short: observation time anchors the dataset, forecast time dictates the predicted trajectory, and diffusion time drives the generative algorithm.

C. Why we convert to a local coordinate frame?

Before defining the trajectory itself, we must decide which coordinate system to use. If the model is trained directly in global map coordinates, it can easily memorize specific locations instead of learning reusable motion patterns[Gao et al., 2020]. For example, a left turn at one intersection and a left turn at another intersection would have completely different global coordinates, even though they are behaviorally similar.

To avoid this, we transform each scene into a local, vehicle agent-centric coordinate frame. We place the focal vehicle agent at the origin $(0, 0)$ and rotate the scene so that the vehicle agent faces the positive x-axis. In this representation, a left turn has a similar geometric pattern no matter where it occurs on the map.

Let $\mathbf{p}_{t_0} \in \mathbb{R}^2$ be the focal vehicle agent's global position and $\theta_{t_0} \in (-\pi, \pi]$ its heading at observation time t_0. For any global coordinate $\mathbf{u}$, the local transformation is

$$\mathrm{Local}(\mathbf{u}) = \mathbf{R}(-\theta_{t_0})(\mathbf{u} - \mathbf{p}_{t_0}),$$

where

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Geometrically, this transformation has two steps:

1. **Translation:** move the scene so that the focal vehicle agent sits at the origin.
2. **Rotation:** rotate the scene so that the focal vehicle agent faces along the positive x-axis.

By applying this transformation to all spatial inputs, each scene is standardized relative to the current pose of the focal vehicle agent. We can visualize this process in Figure 2

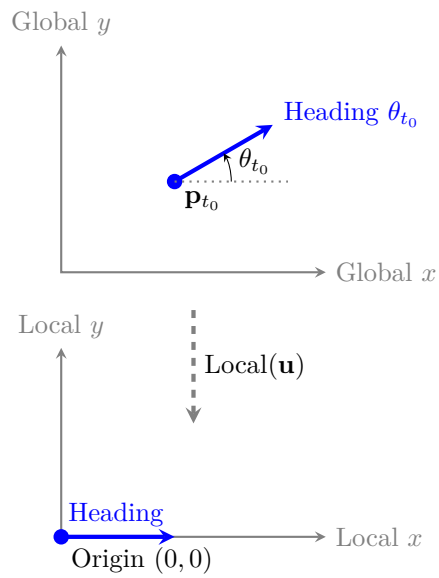

Figure 2: Geometric transformation from the global map coordinates to the vehicle agent-centric local reference frame.

D. How we represent a future trajectory

Once the coordinate system is fixed, we can describe what one future state looks like. At each forecast step $f = 1, \ldots, F$, the local state of an vehicle agent is represented by

$$\mathbf{y}_f = (x_f,\, y_f,\, \sin\theta_f,\, \cos\theta_f) \in \mathbb{R}^4,$$

where (x_f, y_f) are local coordinates and θ_f is the heading angle.

Instead of predicting the raw angle θ_f directly, we predict $(\sin\theta_f, \cos\theta_f)$. This avoids the discontinuity that occurs at the boundary between $-\pi$ and π, which can be visualize in Figure 3.

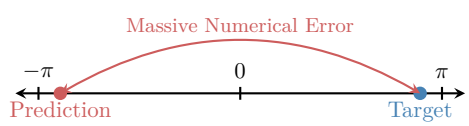

Figure 3: Problem with predicting angle heading directly: points near the boundary $(-\pi, \pi)$ result in large numerical errors despite being physically close.

During inference, we can uniquely recover the predicted heading angle θ_f in the correct mathematical quadrant by applying the two-argument arctangent function, which

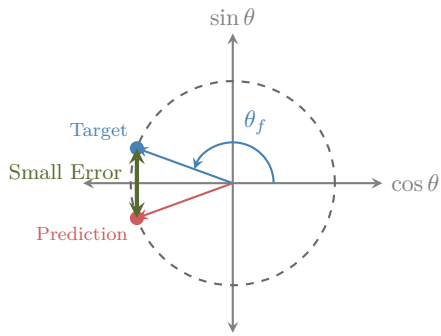

Figure 4: Solution: Unit circle mapping via $(\sin\theta, \cos\theta)$ removes the wrap-around discontinuity and ensures smooth regression.

can be visualized in Figure 4 as the angular sweep from the positive horizontal axis to the coordinate vector:

$$\theta_f = \operatorname{atan2}(\sin\theta_f, \cos\theta_f).$$

For a single vehicle i, stacking all F future states gives the trajectory vector

$$\mathbf{x}_0^{(i)} = \operatorname{vec}(\mathbf{y}_1, \ldots, \mathbf{y}_F) \in \mathbb{R}^{4F}.$$

E. What information does the model receive?

To generate a future, the model must first be told what is currently visible in the scene. We describe this context from the viewpoint of each focal vehicle agent.

For vehicle agent $i \in \{1, \ldots, N_{\text{obj}}\}$, we define the local scene context as

$$\mathcal{C}_i = \{\mathbf{H}_i, \mathbf{N}_i, \mathbf{M}_i, \mathbf{S}_i\}.$$

Each term has a simple interpretation:

- **History ($\mathbf{H}_i$):** the recent motion history of vehicle agent i over the past $H = 11$ steps, expressed in the vehicle agent's current local frame.
- **Neighbors ($\mathbf{N}_i$):** the states of the $K = 6$ nearest surrounding vehicle agents, also transformed into the local frame of vehicle agent i.
- **Map features ($\mathbf{M}_i$):** a compact descriptor of nearby lane and road geometry relative to the focal vehicle agent.
- **Static attributes ($\mathbf{S}_i$):** fixed vehicle agent properties such as object type and physical size.

This representation gives the model four different kinds of information: what the vehicle agent has been doing, who is nearby, what the road looks like, and what kind of object is being predicted.

F. Putting it all together

So far, it is helpful to think about one vehicle agent at a time. But traffic is an interactive system, so the final model must predict all vehicle agents together.

Let N_{obj} be the number of valid vehicle agents in the scene. Stacking the future trajectories of all vehicle agents gives the full future scene tensor

$$\mathbf{X}_0 \in \mathbb{R}^{N_{\text{obj}} \times F \times 4}.$$

Our generative task is therefore to learn the conditional distribution

$$p(\mathbf{X}_0 \mid \mathcal{C}).$$

In the diffusion model, this tensor is updated *all at once* rather than one future timestep at a time, this can be visualized in Figure 5. For computation, we flatten it into a single vector

$$\mathbf{x}_0 = \operatorname{vec}(\mathbf{X}_0) \in \mathbb{R}^{N_{\text{obj}} \times F \times 4}.$$

This vectorized form lets the model refine the future motion of all vehicle agents jointly at each diffusion step. After denoising is complete, the vector is reshaped back into the tensor form $[N_{\text{obj}}, F, 4]$.

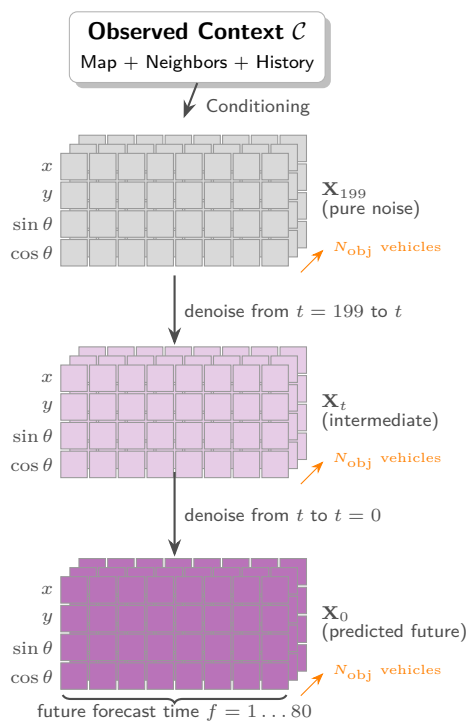

Figure 5: All-at-once diffusion: algorithmic time t denoises a *single 3D tensor* $\mathbf{X}_t$ containing the joint future trajectories for all N_{obj} vehicle agents over physical time $f = 1 \ldots 80$.

III. Background: Denoising Diffusion Probabilistic Models (DDPM)

Diffusion models are a class of generative models inspired by non-equilibrium thermodynamics [Weng, 2021]. The core intuition follows a familiar physical process: a drop of ink dispersing in water until its original structure is lost to diffusion. While reversing this dispersion is physically impossible, a neural network can be trained to learn the mathematical function required to undo this process. Consequently, a diffusion model operates via two distinct stages: the forward process and the reverse process.

A. The Forward Process

The forward process is the simpler of the two stages. Given a clean data point $\mathbf{x}_0 \in \mathbb{R}^d$, where d denotes the dimensionality of the data representation, we gradually destroy its structure by iteratively adding small amounts of Gaussian noise to create a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$. This controlled corruption follows a recursive step:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t, \tag{III.1}$$

where $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is an independent standard Gaussian noise draw, and $\boldsymbol{\beta}_t \in (0,1)$ is a scalar parameter dictating the variance schedule. The sequence $\beta_1, \beta_2, \ldots, \beta_T$ is predetermined, often linearly increasing, which means more noise is added as t increases.

Since the update is a linear transformation of Gaussian noise, the conditional distribution is Gaussian with the mean shown in Equation (III.2) and variance shown in Equation (III.3):

$$\begin{aligned}
\mathbb{E}[\mathbf{x}_t \mid \mathbf{x}_{t-1}] &= \mathbb{E}[\sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t] \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathbb{E}[\boldsymbol{\epsilon}_t] \quad \text{(By Linearity)} \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \mathbf{0} \qquad\qquad (\mathbb{E}[\boldsymbol{\epsilon}_t] = \mathbf{0}) \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1}.
\end{aligned} \tag{III.2}$$

Next, we calculate the conditional variance. Because $\mathbf{x}_{t-1}$ is treated as a given constant, its variance is strictly zero. When we pull the scalar multiplier of the noise term out of the variance operator, we must square it:

$$\begin{aligned}
\mathrm{Var}(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \mathrm{Var}(\sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t) \\
&= \underbrace{\mathbf{0}}_{\mathrm{Var}(\mathrm{const})=0} + \underbrace{(\sqrt{\beta_t})^2\mathrm{Var}(\boldsymbol{\epsilon}_t)}_{\mathrm{Var}(cX)=c^2\mathrm{Var}(X)} \\
&= \beta_t\mathbf{I} \quad (\text{since } \mathrm{Var}(\boldsymbol{\epsilon}_t) = \mathbf{I}).
\end{aligned} \tag{III.3}$$

A fundamental statistical rule states that any linear transformation of a Gaussian variable results in another Gaussian variable. Therefore, $\mathbf{x}_t$ must be normally distributed. This gives us the formal transition probability:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \;=\; \mathcal{N}\Big(\mathbf{x}_t;\; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\; \beta_t\mathbf{I}\Big). \tag{III.4}$$

A key property that makes training efficient is that we do not need to apply this step repeatedly to obtain $\mathbf{x}_t$ from $\mathbf{x}_0$. Because the sum of independent Gaussian variables remains Gaussian, all steps can be collapsed into a single closed-form expression. We introduce $\alpha_t = 1-\beta_t$ to represent the signal retained at each step, allowing us to rewrite the state directly:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\bar{\boldsymbol{\epsilon}}_t, \tag{III.5}$$

where $\bar{\alpha}_t = \prod_{i=1}^{t}\alpha_i$ is the cumulative product of the retained signal, and $\bar{\boldsymbol{\epsilon}}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the combined standard Gaussian noise resulting from the linear addition of all previous noise steps.

This gives us the probability of observing the noisy state $\mathbf{x}_t$ given only the original clean data $\mathbf{x}_0$:

$$p(\mathbf{x}_t \mid \mathbf{x}_0) \;=\; \mathcal{N}\big(\mathbf{x}_t;\; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\; (1-\bar{\alpha}_t)\mathbf{I}\big). \tag{III.6}$$

This formula has a clean geometric interpretation. The state $\mathbf{x}_t$ is a weighted mixture of the original signal and pure noise. Think of it as a cross-fade, where the first term represents how much of the original signal remains, and the second term fills in the rest with noise. By the final timestep T, $\bar{\alpha}_T \approx 0$ and all structures in the original data have been destroyed, leaving $\mathbf{x}_T$ as Gaussian noise.

B. The Reverse Process

While the forward process is fixed and requires no learning, the reverse process must be learned. The goal is to train a neural network to approximate the reverse conditional distribution $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, so that we can start from pure Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively denoise back to a clean trajectory $\mathbf{x}_0$.

Directly computing the true reverse distribution $p(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is hard; we cannot guess where a noisy sample came from without more information. To get around this during training, we use the original clean data $\mathbf{x}_0$ as a guide. By applying Bayes Theorem to our known forward process and the clean starting point $\mathbf{x}_0$, we can map three interacting variables to transform an otherwise hard problem into a simple, easy-to-calculate Gaussian. First, let's review the Chain Rule of Probability:

$$p(A, B, C) = p(A \mid B, C)\, p(B \mid C)\, p(C). \tag{III.7}$$

Rewrite Equation (III.7) to isolate $p(A \mid B, C)$:

$$p(A \mid B, C) = \frac{p(A, B, C)}{p(B \mid C)p(C)}. \tag{III.8}$$

By symmetry, we also know:

$$\begin{aligned} p(A,B,C) &= p(B,A,C) \\ &= p(B \mid A,C)p(A \mid C)p(C). \end{aligned} \tag{III.9}$$

Replacing the numerator, $p(A,B,C)$ from Equation (III.8) with Equation (III.9) gives:

$$\begin{aligned} p(A \mid B,C) &= \frac{p(B \mid A,C)p(A \mid C)p(C)}{p(B \mid C)p(C)} \\ &= \frac{p(B \mid A,C)p(A \mid C)}{p(B \mid C)}. \end{aligned} \tag{III.10}$$

We can apply this exact probabilistic rule to our diffusion variables by setting $A = \mathbf{x}_{t-1}$, $B = \mathbf{x}_t$, and $C = \mathbf{x}_0$. This gives us the explicit form of our reverse process posterior:

$$\underbrace{p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)}_{\text{Reverse Process (posterior)}} = \frac{\overbrace{p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)}^{\text{Forward Step (likelihood)}} \; \overbrace{p(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}^{\text{Prior}}}{\underbrace{p(\mathbf{x}_t \mid \mathbf{x}_0)}_{\text{Normalizing Constant}}}. \tag{III.11}$$

We can simplify, since the forward diffusion process is a Markov chain. This means that once the current state $\mathbf{x}_{t-1}$ is known, the next state $\mathbf{x}_t$ no longer depends directly on the original clean sample $\mathbf{x}_0$:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}). \tag{III.12}$$

So the posterior of Equation (III.11) becomes:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{p(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, p(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{x}_0)}. \tag{III.13}$$

Next, notice that the denominator $p(\mathbf{x}_t \mid \mathbf{x}_0)$ does not depend on the variable we are solving for, namely $\mathbf{x}_{t-1}$. It only serves to normalize the distribution so that it integrates to one. Therefore, if we only care about the shape of the posterior as a function of $\mathbf{x}_{t-1}$, we can ignore this term as a constant of proportionality:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto p(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, p(\mathbf{x}_{t-1} \mid \mathbf{x}_0). \tag{III.14}$$

Now we substitute the two Gaussian terms from the forward process:

$$\begin{aligned} p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\, \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \\ p(\mathbf{x}_{t-1} \mid \mathbf{x}_0) &= \mathcal{N}\Big(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, \mathbf{x}_0, (1-\bar{\alpha}_{t-1})\mathbf{I}\Big). \end{aligned} \tag{III.15}$$

This gives:

$$\begin{aligned} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto\; & \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\, \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \\ & \times \mathcal{N}\Big(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, \mathbf{x}_0, (1-\bar{\alpha}_{t-1})\mathbf{I}\Big). \end{aligned} \tag{III.16}$$

At this point, the key observation is simple: both factors are Gaussians in the same unknown variable $\mathbf{x}_{t-1}$. The first term says that $\mathbf{x}_{t-1}$ should be a value that could plausibly produce the observed $\mathbf{x}_t$ in one forward noising step. The second says that $\mathbf{x}_{t-1}$ should also remain consistent with the clean sample $\mathbf{x}_0$. The posterior is therefore the compromise that satisfies both constraints at once.

To see this mathematically, we write out the exponents of both Gaussian distributions:

$$\begin{aligned} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto \exp\Bigg(& -\frac{1}{2\beta_t} \left\| \mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_{t-1} \right\|^2 \\ & -\frac{1}{2(1-\bar{\alpha}_{t-1})} \left\| \mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 \right\|^2 \Bigg). \end{aligned} \tag{III.17}$$

Expanding the two quadratic terms and regrouping everything involving $\mathbf{x}_{t-1}$ produces a standard quadratic form. Completing the square reveals that this expression is Gaussian in $\mathbf{x}_{t-1}$, from which the posterior mean and variance can be read off directly:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \sigma_t^2 \mathbf{I}). \tag{III.18}$$

The resulting mean and variance are

$$\begin{aligned} \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) &= \underbrace{\frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0}_{\text{pull toward clean } \mathbf{x}_0} + \underbrace{\frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t}_{\text{anchor to noisy } \mathbf{x}_t}, \\ \sigma_t^2 &= \underbrace{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}_{\text{scaled forward variance}}. \end{aligned} \tag{III.19}$$

This mean has a very natural interpretation: it is a weighted average of two pieces of information. One term pulls the sample back toward the clean trajectory $\mathbf{x}_0$, while the other keeps it close to the currently observed noisy state $\mathbf{x}_t$. Early in the reverse process, the model stays close to $\mathbf{x}_t$ and makes only small corrections. As the noise level decreases, the mean shifts more strongly toward $\mathbf{x}_0$.

C. DDPM Inference Procedure

The difficulty during generation is that the clean sample $\mathbf{x}_0$ is unknown. Although the posterior distribution above is exact, we cannot evaluate it directly during inference.

DDPM resolves this limitation by predicting the noise added at step t rather than directly predicting $\mathbf{x}_0$ or the posterior mean. Because the forward process has a closed form, we can rearrange Equation (III.5) to solve for $\mathbf{x}_0$:

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}}. \tag{III.20}$$

If a neural network can accurately predict the added noise $\boldsymbol{\epsilon}$ from $(\mathbf{x}_t, t)$, then it implicitly recovers $\mathbf{x}_0$ as well. Substituting this expression for $\mathbf{x}_0$ into the posterior mean removes the unknown clean sample entirely. This yields:

$$\tilde{\boldsymbol{\mu}}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right). \quad \text{(III.21)}$$

This turns the reverse step into the following distribution:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big). \quad \text{(III.22)}$$

In other words, once the model can estimate how much noise is present in $\mathbf{x}_t$, it can take one denoising step backward.

Algorithm 1 Inference Procedure

1: Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Algorithm 1 translates the learned reverse process into a sequential generation loop. The procedure begins by drawing a pure noise sample from a standard Gaussian distribution. Inside the loop, the algorithm samples from the target Gaussian distribution using a shift and scale operation mathematically identical to the reparameterization trick [Kingma and Welling, 2013]. The procedure computes the exact deterministic mean via the neural network and completes the sample by adding standard Gaussian noise $\mathbf{z}$ scaled by the standard deviation σ_t. For the final step, we set the noise term to zero to ensure the output is the clean data point.

D. DDPM Training Procedure

The training procedure is to minimize the objective function of the neural network by making the predicted noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ as close as possible to the real injected noise $\boldsymbol{\epsilon}$. This is achieved using a simplified mean squared error objective:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}, t} \left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2 \right]. \quad \text{(III.23)}$$

In practice, the network parameters are updated via first-order stochastic optimization techniques such as AdamW, or Stochastic Gradient Descent.

Algorithm 2 Training Procedure

1: **repeat**
2: Sample clean trajectory $\mathbf{x}_0 \sim p(\mathbf{x}_0)$
3: Sample timestep $t \sim \text{Uniform}(\{1, 2, \ldots, T\})$
4: Sample Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient step of Equation (III.23) w.r.t. θ: $\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \boldsymbol{\epsilon}, t \right) \right\|^2$
6: **until** converged

Algorithm 2 describes the complete training procedure, where it begins by drawing a clean data point $\mathbf{x}_0$ from the true distribution and picking a random timestep t. After sampling a standard Gaussian noise vector $\boldsymbol{\epsilon}$, we generate a noisy version of the input. The neural network then predicts the specific noise vector applied to the data, and we take a gradient step with respect to θ to penalize any discrepancy between the actual noise and the predicted noise to improve the neural network weights.

IV. Methods

A. Classifier-Free Guidance

The DDPM in Section II can generate realistic trajectories, but we also want those trajectories to match the scene context $\mathbf{c}$. Classifier-Free Guidance (CFG) [Ho and Salimans, 2021] achieves this by evaluating the same denoising network twice at the same noisy sample $\mathbf{x}_t$: once without context, and once with context.

Recall from Equation (III.5) that $\mathbf{x}_t$ is a noisy version of the clean trajectory $\mathbf{x}_0$. Given $(\mathbf{x}_t, t)$, the network produces

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}_\emptyset) \quad \text{and} \quad \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}),$$

where $\mathbf{c}_\emptyset$ is empty set for the context embedding. This implies that the first prediction is *unconditional*, while the second is *conditional* on the scene.

The key idea is that the difference between these two predictions captures how the scene should change the denoising direction. During sampling, we therefore use

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, \mathbf{c}) = \underbrace{(1-w)\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing)}_{\text{suppressed unconditional score}} + \underbrace{w\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c})}_{\text{amplified conditional score}}, \quad \text{(IV.1)}$$

where $w \geq 0$ is the guidance scale. If $w = 0$, generation is unconditional; if $w = 1$, we recover the usual conditional model; and if $w > 1$, the model follows the scene more strongly. In our experiments, we use $w = 1.2$.

For variance-preserving diffusion, combining noise predictions as in Equation (IV.1) is equivalent to combining

scores, so CFG can be applied directly in the DDPM noise-prediction setting.

A.0.1 Training. To train one network to perform both unconditional and conditional denoising, we randomly drop the context during training:

$$\mathbf{c}' = \begin{cases} \mathbf{c}_\emptyset & \text{with probability } p_{\text{drop}}, \\ \mathbf{c} & \text{otherwise.} \end{cases} \tag{IV.2}$$

The training objective is the same noise-prediction loss as in Equation (III.23), except that the network receives $\mathbf{c}'$:

$$\mathcal{L}_{\text{cfg}} = \mathbb{E}_{\mathbf{x}_0,\boldsymbol{\epsilon},t}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}')\|^2\right]. \tag{IV.3}$$

A.0.2 Inference. At test time, each reverse step evaluates the network twice: once with $\mathbf{c}_\emptyset$ and once with $\mathbf{c}$. The two predictions are combined using Equation (IV.1), and the guided noise estimate $\tilde{\boldsymbol{\epsilon}}_\theta$ is used in the reverse DDPM update. Since this doubles the number of network evaluations per step, we sample with 50 reverse steps instead of the full $T = 200$.

B. From Model Output to Final Rollouts

The diffusion model produces a trajectory proposal, but that proposal is not guaranteed to be smooth, physically realistic, or safe. We therefore compare four decoding strategies. These can be read as a progression from the simplest possible baseline to the most constrained model.

B.1 Model 1: Constant Velocity Baseline

The constant velocity baseline assumes that an vehicle agent continues to move with the exact velocity and heading observed at the current time. Let the initial position be (x_0, y_0), the initial heading be θ_0, and the initial speed be v_0. For a prediction horizon T and time step Δt, the future states are extrapolated purely linearly:

$$\begin{aligned} x_{t+1} &= x_t + v_0 \cos(\theta_0)\Delta t, \\ y_{t+1} &= y_t + v_0 \sin(\theta_0)\Delta t, \\ \theta_{t+1} &= \theta_0, \\ v_{t+1} &= v_0. \end{aligned} \tag{IV.4}$$

This model is easy to interpret and serves as a strong baseline for straight-line motion, but it cannot capture turning, acceleration changes, or interactions.

B.2 Model 2: Raw Geometric Decoding

The next decoder directly transforms the diffusion output from the local frame back into the global frame. If the model predicts local displacements $(\Delta x_t, \Delta y_t)$ and heading representation $(\sin\Delta\theta_t, \cos\Delta\theta_t)$, then

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \mathbf{R}(\theta_0)\begin{bmatrix} \Delta x_t \\ \Delta y_t \end{bmatrix}, \tag{IV.5}$$

and

$$\theta_t = \text{atan2}(\sin\Delta\theta_t, \cos\Delta\theta_t). \tag{IV.6}$$

This approach is fast, but it treats each future step too independently and can produce physically implausible motion.

B.3 Model 3: Bicycle Kinematics

To enforce temporal consistency, we interpret the diffusion output as control-like motion proposals and roll them out with a kinematic bicycle model [Polack et al., 2017]. With steering angle δ_t, acceleration a_t, and wheelbase L, we update

$$\begin{aligned} x_{t+1} &= x_t + v_t \cos(\theta_t)\Delta t, \\ y_{t+1} &= y_t + v_t \sin(\theta_t)\Delta t, \\ \theta_{t+1} &= \theta_t + \frac{v_t}{L}\tan(\delta_t)\Delta t, \\ v_{t+1} &= v_t + a_t \Delta t. \end{aligned} \tag{IV.7}$$

This produces smoother trajectories, but it still does not explicitly prevent off-road motion or collisions.

B.4 Model 4: Bicycle Kinematics with Safety Constraints

Our most structured decoder keeps the same kinematic update, but adds explicit safety checks. First, we discourage off-road motion using a minimum road-edge clearance:

$$d_{\text{edge}}(x_t, y_t) \geq d_{\min}. \tag{IV.8}$$

Second, for any pair of agents i and j, we enforce a minimum separation:

$$\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \geq r_{ij}. \tag{IV.9}$$

We also bound speed by $0 \leq v_t \leq v_{\max}$. Together, these constraints produce trajectories that are not only smooth but also more physically plausible and safer in the traffic scene.

C. Summary of the complete method

We summarize the complete forecasting pipeline in Figure 6.

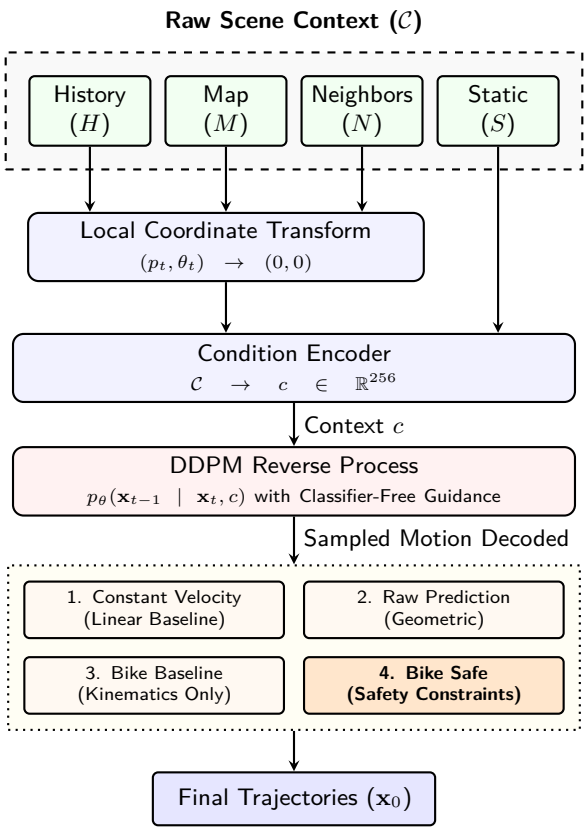

Figure 6: Conditional motion forecasting architecture.

The full model works as follows. We first transform the scene into local coordinates and encode the observed context for each agent. The diffusion model then denoises a joint future trajectory conditioned on these context embeddings, with classifier-free guidance used at inference time to improve scene consistency. Finally, one of the four decoders converts the generated motion into a complete rollout which can be visualized in Figure 7

V. Results and Discussion

With the full system now defined, we evaluate it in the same order in which the model was built. We first ask whether the predicted trajectories stay close to the ground truth. We then ask a second and equally important question: are those trajectories physically realistic and safe?

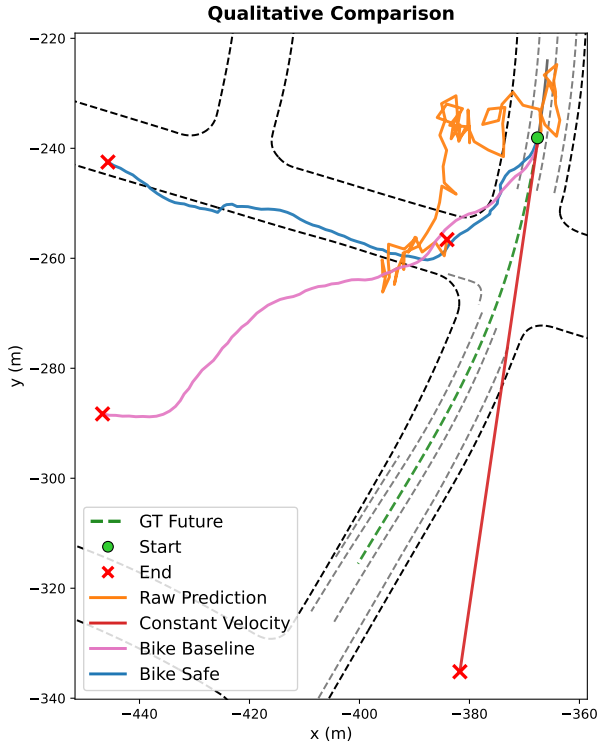

Figure 7: Comparison of four decoding strategies on Scenario 26. The safety-constrained bicycle decoder produces smoother trajectories and fewer off-road violations.

A. Trajectory Accuracy

We first assess how close the predicted paths are to the actual paths taken by the vehicle agents. We measure this using Mean Displacement Error (MDE), which calculates the average Euclidean distance between the predicted and ground truth positions over a set number of future steps T.

$$MDE = \frac{1}{T}\sum_{t=1}^{T}\sqrt{(x_t^{pred} - x_t^{gt})^2 + (y_t^{pred} - y_t^{gt})^2}, \quad \text{(V.1)}$$

where (x_t^{pred}, y_t^{pred}) represents the predicted position at time t and (x_t^{gt}, y_t^{gt}) represents the ground truth position at time t.

As shown in Figure 8, lower MDE values indicate more accurate predictions. The Constant Velocity (CV) baseline performs best overall. This is expected because the CV model assumes continuous speed and direction, which perfectly matches the straight-line nature of most scenarios in our dataset.

When comparing the more complex models, the Legacy decoder and the Bicycle Physics model without safety constraints converge to nearly identical error rates by step 60. However, the Bicycle Physics model equipped

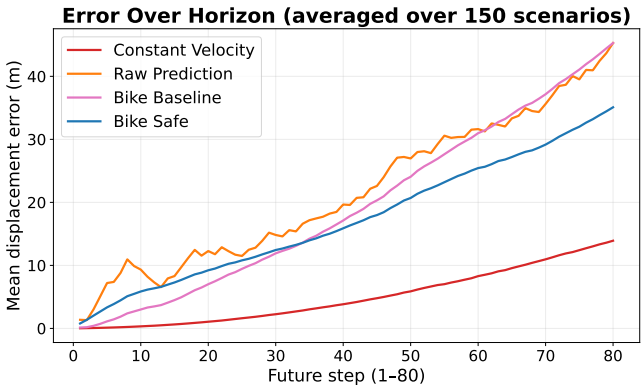

Figure 8: Mean Displacement Error averaged over 150 held-out validation scenarios, each representing a 9-second, multi-vehicle agent traffic interaction.

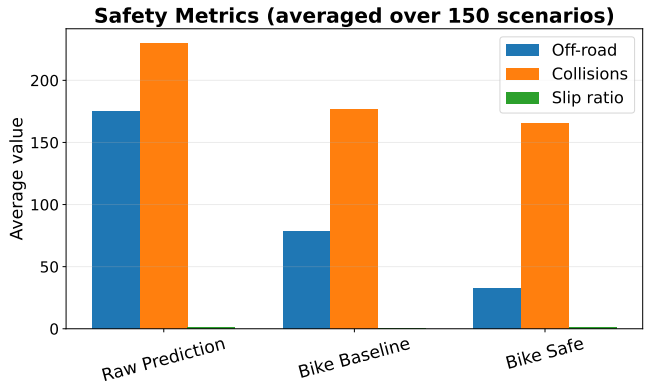

Figure 9: Safety metrics averaged over 150 held-out validation scenarios with $d_{min} = 1.2$ meter, and $r_{ij} = 0.8$ meter.

with full safety constraints consistently outperforms both alternatives across the time horizon.

B. Safety and Physical Realism

Accuracy alone does not guarantee a realistic or safe trajectory. Therefore, we also evaluate how each decoding strategy impacts physical realism using three post-processing safety metrics.

Let M denote the set of evaluated models: the legacy decoder, the bicycle physics model without explicit safety constraints, and the full bicycle physics model with road-bound and interaction-aware corrections. For each model $m \in M$, we evaluate a safety profile defined by the tuple $\mathcal{S}(m) = (O(m), C(m), \Sigma(m))$:

- **Off-road violations** $O(m)$: Measures adherence to drivable space. An event is recorded if the distance to the nearest road edge falls below a clearance threshold $d_{\min} > 0$:

$$d_{\text{edge}}(\mathbf{p}_i(t)) < d_{\min}.$$

- **Collision pairs** $C(m)$: Measures unsafe proximity between vehicle agents. A collision is registered if the distance between vehicle agents i and j violates a safety threshold r_{ij} based on their respective sizes:

$$\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| < r_{ij}.$$

- **Mean slip ratio** $\Sigma(m)$: Quantifies lateral deviation from the forward heading. A smaller slip ratio indicates smoother and more physically realistic vehicle kinematics.

Figure 9 clearly shows that as we added more constraints to decoding the trajectory, we get a safer final result in terms of both off-road and collision.

VI. Limitations

A. CV baseline dominance.

The constant-velocity baseline outperforms all diffusion-based decoders in mean displacement error across the full 80-step horizon. This is expected given the straight-line majority of Waymo scenarios, but highlights that the diffusion model has not yet learned to leverage scene conditioning strongly enough to surpass a trivial predictor on average.

B. Bicycle kinematics alone provide no benefit.

Figure 8 shows that the bicycle physics model without safety constraints converges to nearly identical error as the legacy decoder by step 60. This suggests that kinematic plausibility alone does not improve trajectory accuracy. The road-boundary enforcement and collision avoidance constraints are the critical ingredients that produce the reduction seen in the full model.

C. Absence of external evaluation.

Results are evaluated using internally defined metrics, making it impossible to contextualize performance against published methods. The Waymo Scenario Generation Challenge provides a standardized Realism Meta Metric (RMM) that would allow direct comparison to results such as SimFormer (RMM = 0.6623).

VII. Conclusion and Future Direction

In this project, we developed a context-conditioned diffusion framework for Waymo scenario generation, conditioning on vehicle agent history, neighboring vehicle agents,

map geometry, and static object properties. The model predicts 80-step futures and supports both quantitative evaluation and qualitative visualization of rollouts.

Overall, results show strong short-horizon accuracy and generally realistic trajectories, especially when paired with lightweight post-processing to encourage safety and physical plausibility. Future work could improve scalability and realism by training on a larger portion of the dataset, extending to an autoregressive chunked forecasting scheme that predicts trajectories in segments up to a horizon F, and introducing additional generative components to increase diversity (e.g., a separate diffusion model for vehicle dimensions and randomized initial states).

References

Chen Chen et al. Scenario dreamer: Vectorized latent diffusion for generating driving simulation environments. *arXiv preprint arXiv:2309.13110*, 2023.

Nanxin Chen, Yu Zhang, Heiga Zen, Ron J Weiss, Mohammad Norouzi, and Najim Dehak. Wavegrad: Estimating gradients for waveform generation. In *International Conference on Learning Representations*, 2021.

Scott Ettinger et al. Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Jiyang Gao, Chen Sun, Hang Zhao, Yi Shen, Dragomir Anguelov, Congcong Li, and Cordelia Schmid. Vectornet: Encoding hd maps and agent dynamics from vectorized representation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

Jonathan Ho and Tim Salimans. Classifier free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.

John B Ingraham, Max Baranov, Zak Costello, Karl W Barber, Wujie Wang, Ahmed Ismail, Artem Lebedev, Alex Bennion, Pavel Golubkov, and Philip Bradley. Illuminating protein space with a programmable generative model. *Nature*, 2023.

Diederik P Kingma and Max Welling. Auto encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. Diffwave: A versatile diffusion model for audio synthesis. In *International Conference on Learning Representations*, 2021.

Mengxuan Li et al. Diffscene: Diffusion based safety critical scenario generation for autonomous vehicles. *arXiv preprint arXiv:2309.15103*, 2023.

Shitong Luo, Yufeng Su, Xingang Peng, Sheng Wang, Jian Peng, and Jianzhu Ma. Antigen specific antibody design and optimization with diffusion based generative models. *Advances in Neural Information Processing Systems*, 2022.

Inzaghi Moniaga and Baylor Taylor. Diffusion model for waymo scenario generation. *github.com/InMDev/Diffusion-Model-for-Waymo-Scenario-Generation*, 2025.

Philip Polack, Florent Altché, Brigitte d'Andréa Novel, and Arnaud de La Fortelle. The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles? In *2017 IEEE Intelligent Vehicles Symposium (IV)*, pages 812–818, 2017. doi: 10.1109/IVS.2017.7995816.

Brian L Trippe, Jason Yim, Doug Tischer, David Baker, Tamara Broderick, Regina Barzilay, and Tommi Jaakkola. Diffusion probabilistic modeling of protein backbones in 3d for the motif scaffolding problem. In *International Conference on Learning Representations*, 2023.

Yiyang Wang et al. Versatile behavior diffusion model for smart agent simulation. *arXiv preprint*, 2024.

Lilian Weng. What are diffusion models? *lilianweng.github.io*, Jul 2021.

Gen AI declaration

Generate TikZ diagram, improve the plot readability, explain Code, debug, and write meaningful comment.